SentinelSSH

Analyseur de logs SSH — Sécurité défensive

Stage 4 — Plan des sprints et exécution

Portfolio Project — MVP Development & Execution & Technical Manual Review (Stage 4 - France)

Anil AKER
Holberton School Bordeaux
Projet de fin d'année — Demo Day / Titre RNCP niveau 5
github.com/anil-akr
1er octobre 2026

> **Quatre sprints d'une semaine.** Chaque sprint a un objectif, des tâches estimées et datées, une revue et une rétrospective. À la fin de chacun, une partie de l'application fonctionne et une version est taguée.

Sommaire

1. Organisation de l'équipe pour le développement 3

2. Vue d'ensemble des sprints 4

3. Plan détaillé des sprints 5

4. Exécution des tâches 8

5. Suivi de l'avancement et ajustements 9

6. Revues de sprint et rétrospectives 11

7. Intégration finale et tests de QA (sprint 4) 11

8. Livrables à fournir (liens) 12

9. Préparer la revue technique (MR) 12

Annexe — Journal des décisions 14

Correspondance avec les consignes du Stage 4

Consigne Holberton	Où la trouver
Rôles clés : PM, SCM, QA, rôles techniques	§1
Task 0 — Plan and Define Sprints	§2 et §3
Task 1 — Execute Development Tasks	§4
Task 2 — Monitor Progress and Adjust	§5
Task 3 — Sprint Reviews and Retrospectives	§6
Task 4 — Final Integration and QA Testing	§7
Task 5 — Deliverables (liens)	§8
Task 6 — Technical Manual Review	§9

1. Organisation de l'équipe pour le développement

Le projet étant individuel, Anil AKER tient les quatre rôles. Chaque rôle a des moments précis dans la semaine : c'est ce qui garantit qu'aucun n'est oublié quand le développement accapare l'attention.

1.1 Rôles et responsabilités

Rôle	Responsabilités pendant le Stage 4	Quand
Project Manager (PM)	Planifie chaque sprint, tient le kanban à jour, suit les métriques, décide des ajustements, anime la revue et la rétrospective	Lundi (planification), chaque jour (point quotidien), jeudi (point de mi-sprint), dimanche (revue, rétrospective)
Source Control Manager (SCM)	Fait respecter les branches, relit chaque Pull Request avec la checklist, protège main, fusionne develop dans main et crée les tags	À chaque Pull Request ; dimanche (fusion et tag)
Quality Assurance (QA)	Écrit les tests avec le code, rejoue Postman et la checklist, ouvre les issues de bug, valide avant chaque fusion	Chaque jour, avant chaque fusion ; dimanche (QA du sprint)
Développeur (back-end, base de données, front-end)	Réalise les tâches du sprint en respectant les normes de code et la documentation	Le reste du temps

1.2 Une semaine type de sprint

Jour	Activités
Lundi	**Planification** (30 min) : objectif du sprint, choix et estimation des tâches, création des issues dans la milestone du sprint. Puis développement.
Mardi → samedi	**Point quotidien écrit** (5 min) : hier, aujourd'hui, blocages. Développement, Pull Requests, revue et tests avant chaque fusion.
Jeudi	**Point de mi-sprint** : si moins de la moitié des points sont terminés, le PM ajuste le périmètre (§5.3).
Dimanche	QA du sprint, **revue de sprint** (démonstration), **rétrospective** (15 min), fusion de develop dans main et tag, point hebdomadaire envoyé à l'équipe pédagogique.

1.3 Outils

Besoin	Outil	Utilisation
Suivi des tâches	GitHub Projects	Kanban À faire / En cours / En revue / Terminé ; une milestone par sprint avec sa date de fin
Tâches et bugs	GitHub Issues	Une issue par tâche (modèle « Tâche ») ou par bug (modèle « Bug ») ; labels de priorité et de gravité
Code et revues	Git, GitHub, Pull Requests	Branches main / develop / feature ; modèle de PR avec checklist
Tests	pytest, pytest-cov, Postman, pycodestyle, ESLint, GitHub Actions	Tests automatiques à chaque push ; tests manuels de l'API et du parcours
Documentation du suivi	Dossier docs/sprints du dépôt	Plan des sprints, points quotidiens, revues, rétrospectives : tout est versionné et partageable par lien

GitHub Projects remplace Trello ou Jira : les tâches, le code et les Pull Requests sont au même endroit, et une issue se ferme automatiquement quand la PR qui la résout est fusionnée (« Closes #12 »).

2. Vue d'ensemble des sprints

2.1 Objectifs et incréments

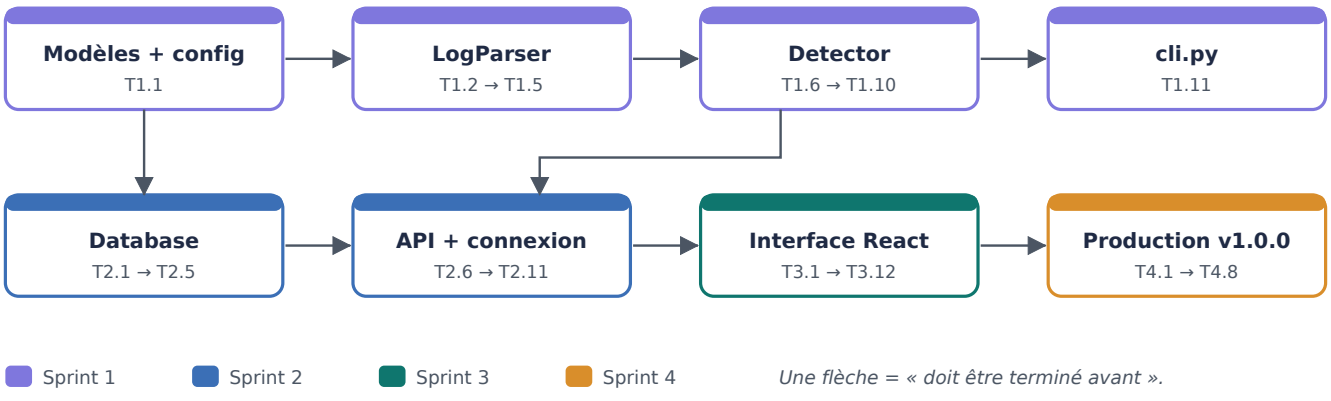
Chaque sprint dure une semaine, du lundi au dimanche. L'**incrément** est ce qui fonctionne à la fin du sprint.

Sprint	Dates	Objectif	Points	Jalon
Sprint 0 Préparation	2 → 4 oct. 2026	Le dépôt, les outils de suivi et l'environnement de travail sont prêts.	—	J3 — 04/10
Sprint 1 Moteur d'analyse	lun. 5 → dim. 11 oct.	Le moteur d'analyse repère les 3 types d'attaques sur des fichiers d'essai et s'utilise en ligne de commande.	26	J4 — 11/10
Sprint 2 Base de données, API et connexion	lun. 12 → dim. 18 oct.	Les résultats sont enregistrés et accessibles par l'API ; seul l'administrateur connecté y a accès.	29	J5 — 18/10
Sprint 3 Interface React	lun. 19 → dim. 25 oct.	L'application complète fonctionne dans le navigateur, sur ordinateur et sur téléphone.	25	J6 — 25/10
Sprint 4 Intégration finale, QA et mise en service	lun. 26 oct. → dim. 1er nov.	La version 1.0 est stable, testée, documentée et lancée en production.	22	J7 — 01/11

Les points mesurent l'effort estimé d'une tâche (1 = petite, 2 = moyenne, 3 = difficile, 5 = très grosse). Les tâches « Could » ne sont pas comptées dans les points prévus : elles ne sont faites que s'il reste du temps.

2.2 Dépendances principales

Certaines parties ne peuvent commencer que lorsque d'autres sont terminées. Ce chemin fixe l'ordre des sprints.



Le moteur d'analyse (sprint 1) ne dépend de rien : c'est pour cela qu'il vient en premier. L'interface React a besoin d'une API qui fonctionne : elle vient en troisième.

3. Plan détaillé des sprints

Chaque user story du Stage 3 est découpée en tâches. Colonnes : **US** = user story concernée ; **Priorité** MoSCoW ; **Pts** = estimation ; **Dépend de** = tâche à terminer avant ; **Rôle** = casquette principale.

Sprint 0 — Préparation · 2 → 4 oct. 2026

Objectif : Le dépôt, les outils de suivi et l'environnement de travail sont prêts.
Incrément : Dépôt GitHub public avec main et develop, kanban, labels, milestones, modèles d'issue et de PR

ID	Tâche	US	Priorité	Pts	Dépend de	Échéance	Rôle
T0.1	Créer le dépôt public sentinelssh, les branches main et develop, protéger main	—	**Must**	1	—	ven. 2/10	SCM
T0.2	Configurer GitHub Projects (kanban), les labels, les milestones Sprint 1 à 4, les modèles d'issue et de PR	—	**Must**	1	T0.1	sam. 3/10	PM, SCM
T0.3	Créer les issues du sprint 1 à partir de ce plan	—	**Must**	1	T0.2	sam. 3/10	PM
T0.4	Environnement : venv Python, Node.js LTS, structure des dossiers, .gitignore, README initial	—	**Must**	1	T0.1	dim. 4/10	Dev

Sprint 1 — Moteur d'analyse · lun. 5 → dim. 11 oct.

Objectif : Le moteur d'analyse repère les 3 types d'attaques sur des fichiers d'essai et s'utilise en ligne de commande.
Incrément : LogParser, Detector, cli.py et leurs tests unitaires ; tag v0.1.0

ID	Tâche	US	Priorité	Pts	Dépend de	Échéance	Rôle
T1.1	config.py et models.py (dataclasses Event et SuspectIP)	US1	**Must**	2	T0.4	lun. 5/10	Dev
T1.2	LogParser.parse_line : lignes Failed et Accepted, « invalid user », IPv6, date (année 2000)	US1	**Must**	3	T1.1	mar. 6/10	Dev
T1.3	LogParser.parse_text : compteurs de lignes lues et ignorées	US1	**Must**	1	T1.2	mar. 6/10	Dev
T1.4	Mini-journaux d'essai (tests/samples) et demo.log	US2, US3	**Must**	2	—	mar. 6/10	QA
T1.5	Tests unitaires de LogParser	US1	**Must**	2	T1.2, T1.4	mer. 7/10	QA
T1.6	Detector : règle R1 (attaque persistante)	US2	**Must**	2	T1.1	mer. 7/10	Dev
T1.7	Detector : règle R2 (force brute rapide, fenêtre de 60 s)	US2	**Must**	3	T1.6	jeu. 8/10	Dev

ID	Tâche	US	Priorité	Pts	Dépend de	Échéance	Rôle
T1.8	Detector : règle R3 (compromission possible)	US3	**Must**	3	T1.6	ven. 9/10	Dev
T1.9	Detector : gravité, tri, comptes les plus visés	US2	**Must**	2	T1.7, T1.8	ven. 9/10	Dev
T1.10	Tests unitaires de Detector	US2, US3	**Must**	3	T1.9	sam. 10/10	QA
T1.11	cli.py : analyse d'un fichier et tableau texte	US1	**Must**	2	T1.3, T1.9	sam. 10/10	Dev
T1.12	Essai sur l'échantillon Loghub (2 000 lignes)	—	Should	1	T1.11	dim. 11/10	QA
T1.13	Tutoriel React officiel, environ 1 h par jour (préparation du sprint 3)	—	Should	—	—	en continu	Dev
	Total des points prévus (hors Could)			**26**			

Sprint 2 — Base de données, API et connexion · lun. 12 → dim. 18 oct.

Objectif : Les résultats sont enregistrés et accessibles par l'API ; seul l'administrateur connecté y a accès.

Incrément : Database, les 8 routes de l'API, la connexion, les tests d'intégration et la CI ; tag v0.2.0

ID	Tâche	US	Priorité	Pts	Dépend de	Échéance	Rôle
T2.1	schema.sql : 3 tables, clés, contraintes	US5	**Must**	2	—	lun. 12/10	Dev
T2.2	Database : connect (clés étrangères) et init_db	US5	**Must**	1	T2.1	lun. 12/10	Dev
T2.3	Database : enregistrer, lister, lire et supprimer une analyse	US5, US8	**Must**	3	T2.2	mar. 13/10	Dev
T2.4	Database : récidivistes, lecture et création du compte	US7, US0	**Must**	2	T2.2	mar. 13/10	Dev
T2.5	Tests unitaires de Database (cascade, récidivistes)	US5, US7	**Must**	2	T2.3, T2.4	mer. 14/10	QA
T2.6	app.py : configuration, .env, taille maximale, erreurs en JSON	US1	**Must**	2	—	mer. 14/10	Dev
T2.7	Connexion : login, logout, me, login_required, commande create-admin	US0	**Must**	3	T2.4, T2.6	jeu. 15/10	Dev
T2.8	Routes des analyses : envoi et vérifications, liste, détail, suppression	US1, US5, US8	**Must**	5	T2.3, T2.7	ven. 16/10	Dev
T2.9	Route des récidivistes et commande de blocage	US4, US7	**Must**	1	T2.8	ven. 16/10	Dev
T2.10	Tests d'intégration de l'API (test_api.py)	US0, US1	**Must**	3	T2.9	sam. 17/10	QA

ID	Tâche	US	Priorité	Pts	Dépend de	Échéance	Rôle
T2.11	Collection Postman des 8 routes	—	Should	1	T2.9	sam. 17/10	QA
T2.12	CI GitHub Actions : pycodestyle, pytest, couverture	—	**Must**	2	T2.5	dim. 18/10	SCM
T2.13	Création du projet Vite et premier appel à l'API	US5	Should	2	T2.8	dim. 18/10	Dev
	Total des points prévus (hors Could)			**29**			

Sprint 3 — Interface React · lun. 19 → dim. 25 oct.

Objectif : L'application complète fonctionne dans le navigateur, sur ordinateur et sur téléphone.
Incrément : Les 4 pages et les 10 composants, ESLint et construction du front dans la CI ; tag v0.3.0

ID	Tâche	US	Priorité	Pts	Dépend de	Échéance	Rôle
T3.1	App.jsx (routes) et api.js (appels, gestion du 401)	US0	**Must**	3	T2.13	lun. 19/10	Dev
T3.2	LoginPage, ProtectedRoute et Header	US0	**Must**	3	T3.1	mar. 20/10	Dev
T3.3	UploadForm	US1	**Must**	2	T3.1	mar. 20/10	Dev
T3.4	HistoryTable avec suppression	US5, US8	**Must**	2	T3.1	mer. 21/10	Dev
T3.5	AnalysisPage et SummaryCards	US2	**Must**	2	T3.1	mer. 21/10	Dev
T3.6	SuspectsTable, SeverityBadge et CopyButton	US2, US3, US4	**Must**	3	T3.5	jeu. 22/10	Dev
T3.7	RecidivistsTable	US7	Should	1	T3.4	jeu. 22/10	Dev
T3.8	TopIpsChart (Recharts)	US6	Should	2	T3.5	ven. 23/10	Dev
T3.9	PrivacyPage	US10	Should	1	—	ven. 23/10	Dev
T3.10	CSS responsive et accessibilité	US9	Should	3	T3.6	sam. 24/10	Dev
T3.11	ESLint et construction du front dans la CI	—	**Must**	1	T2.12	sam. 24/10	SCM
T3.12	Premier passage complet de la checklist de bout en bout	Toutes	**Must**	2	T3.10	dim. 25/10	QA
	Total des points prévus (hors Could)			**25**			

Sprint 4 — Intégration finale, QA et mise en service · lun. 26 oct. → dim. 1er nov.

Objectif : La version 1.0 est stable, testée, documentée et lancée en production.
Incrément : Version finale v1.0.0, README complet, preuves de tests, environnement de production

ID	Tâche	US	Priorité	Pts	Dépend de	Échéance	Rôle
T4.1	Flask sert le front construit (dist) et renvoie index.html hors /api	—	**Must**	2	T3.12	lun. 26/10	Dev
T4.2	Pré-production : Gunicorn, .env, base de test	—	**Must**	2	T4.1	lun. 26/10	SCM
T4.3	Robustesse : 655 146 lignes Loghub ; 5 Mo en moins de 5 s	—	**Must**	2	—	mar. 27/10	QA
T4.4	Checklist complète en pré-production et correction des bugs	Toutes	**Must**	5	T4.2	mer. 28/10	QA, Dev
T4.5	Couverture de tests d'au moins 80 % : compléter les tests manquants	—	**Must**	2	—	mer. 28/10	QA
T4.6	README complet : architecture, base, installation, tests, déploiement	—	**Must**	3	—	jeu. 29/10	Dev
T4.7	Docstrings et commentaires du code	—	**Must**	2	—	jeu. 29/10	Dev
T4.8	Fusion dans main, tag v1.0.0, installation en production	—	**Must**	1	T4.4	ven. 30/10	SCM
T4.9	Vidéo de démonstration en production	—	Should	1	T4.8	ven. 30/10	PM
T4.10	Préparation de la revue technique (MR)	—	**Must**	2	T4.8	sam. 31/10	PM
T4.11	Bonus : limitation des tentatives de connexion, export CSV	US11	Could	3	—	si avance	Dev
	Total des points prévus (hors Could)			**22**			

4. Exécution des tâches

4.1 Le trajet d'une tâche

Toutes les tâches suivent le même chemin, de l'issue à la fusion :



Si la CI est rouge ou si la revue trouve un problème : retour à l'étape 3.

4.2 Normes de code et de documentation

- **Python :** style vérifié par pycodestyle ; une docstring pour chaque module, classe et méthode publique ; noms explicites en anglais.
- **JavaScript :** ESLint sans erreur ; un composant par fichier ; toutes les requêtes dans `api.js`.
- **Commentaires :** ils expliquent le *pourquoi* (par exemple l'année 2000 dans le parser), pas ce que le code fait déjà clairement.
- **Sécurité :** requêtes SQL paramétrées uniquement ; aucun secret dans le code ; jamais `dangerouslySetInnerHTML`.
- **Commits :** format Conventional Commits (`feat:`, `fix:`, `test:`, `docs:`, `refactor:`), au moins un par session de travail.

4.3 Revue de code (rôle SCM)

Chaque Pull Request est relue avec la checklist du modèle de PR avant d'être fusionnée dans `develop` :

- La CI est verte (style, tests, couverture, construction du front).
- Le code fait ce que dit l'issue, et rien de plus.
- Les tests couvrent le cas normal et au moins un cas d'erreur.
- Aucun secret, aucune donnée réelle, aucun fichier inutile (base, `.env`, `node_modules`).
- Je peux expliquer chaque ligne à voix haute. Si ce n'est pas le cas, la PR n'est pas fusionnée.

Les PR sensibles (connexion, base de données) sont montrées à un camarade quand c'est possible ; sa relecture est notée dans la PR.

4.4 Tests par la QA pendant le sprint

- **Pendant le développement :** chaque tâche arrive avec ses tests unitaires ou d'intégration.
- **Sprint 2 :** la collection Postman des 8 routes est rejouée après chaque modification de l'API.
- **Sprints 3 et 4 :** la checklist de bout en bout (13 scénarios) est rejouée avant chaque fusion dans `main`.
- **Chaque bug trouvé** devient une issue avec le label `bug` et une gravité.

Définition de « terminé »

- Code fusionné dans develop par une PR relue.
- Tests écrits et verts dans la CI.
- Checklist concernée validée.
- Issue fermée et documentation à jour si besoin.

5. Suivi de l'avancement et ajustements

5.1 Point quotidien (stand-up)

Chaque jour de travail, 5 minutes, à l'écrit, dans `docs/sprints/sprint-N-standups.md` :

- **Hier :** ce qui a été terminé.
- **Aujourd'hui :** ce qui est prévu.
- **Blocages :** ce qui empêche d'avancer, et la personne à qui demander de l'aide si le blocage dure plus d'une demi-journée.

5.2 Métriques suivies

Métrique	Définition	Objectif
Vélocité	Nombre de points des tâches terminées pendant le sprint	Stable d'un sprint à l'autre (environ 25)
Taux de réalisation	Points terminés ÷ points prévus	Au moins 90 % des tâches Must
Bugs	Nombre de bugs ouverts et corrigés ; taux de résolution = corrigés ÷ ouverts	Aucun bug bloquant ouvert en fin de sprint
Couverture de tests	Part du code d'analyse exécutée par les tests (pytest-cov)	Au moins 80 % à la fin du sprint 4
État de la CI	Dernier passage de GitHub Actions sur develop	Toujours vert en fin de journée

Tableau de suivi (rempli à chaque revue de sprint)

Sprint	Points prévus	Points terminés	Réalisation	Bugs ouverts / corrigés	Couverture	CI
Sprint 1	26					
Sprint 2	29					
Sprint 3	25					
Sprint 4	22					

5.3 Règles d'ajustement

- **Point de mi-sprint (jeudi) :** si moins de la moitié des points sont terminés, les tâches Should du sprint passent au sprint suivant.
- **Tâche qui déborde :** si une tâche dépasse le double de son estimation, elle est découpée en deux issues ; la cause est notée pour la rétrospective.
- **Bug bloquant :** il est corrigé avant toute nouvelle fonctionnalité.
- **Retard global :** ordre de coupe fixé à l'avance : 1) bonus, 2) graphique, 3) récidivistes. La connexion, la détection, la base, l'API, le tableau et les tests ne sont jamais coupés.
- **Chaque ajustement** est noté dans la revue du sprint et, s'il change le périmètre, dans le journal des décisions.

5.4 Suivi des bugs

Gravité (label)	Définition	Délai de correction
`bloquant`	Une fonction Must ne marche pas, ou faille de sécurité	Immédiat, avant toute autre tâche
`majeur`	Une fonction marche mal, mais un contournement existe	Avant la fin du sprint en cours
`mineur`	Détail d'affichage ou de confort	Sprint 4 ; sinon noté comme limite connue

Chaque bug utilise le modèle d'issue « Bug » : étapes pour le reproduire, résultat attendu, résultat obtenu, gravité. Sa correction passe par une branche `fix/…` et une PR qui le ferme.

6. Revues de sprint et rétrospectives

6.1 Revue de sprint (dimanche)

But : montrer ce qui fonctionne. Chaque revue est écrite dans `docs/sprints/sprint-N-review.md` :

- Objectif du sprint et verdict : atteint, partiellement atteint ou non atteint.
- Tâches terminées et non terminées (avec la raison), liens vers les PR.
- **Démonstration** de l'incrément à une partie prenante (camarade, formateur) ou enregistrée en courte vidéo ; les retours reçus sont notés.
- Métriques du sprint (§5.2) et tag de version créé.

6.2 Rétrospective (dimanche, 15 minutes)

But : améliorer la façon de travailler. Écrite dans `docs/sprints/sprint-N-retro.md`, autour de trois questions :

- Qu'est-ce qui a bien marché pendant ce sprint ?
- Quelles difficultés ai-je rencontrées ?
- Quelle action concrète je mets en place au prochain sprint ? (une ou deux au maximum)

La rétrospective suivante commence par vérifier si les actions précédentes ont été appliquées.

7. Intégration finale et tests de QA (sprint 4)

7.1 Tests d'intégration de bout en bout

Ce qu'on vérifie	Comment
Le front appelle correctement toutes les routes de l'API	Parcours complet dans le navigateur ; onglet Réseau des outils de développement de Chrome (codes HTTP, JSON reçus)
La connexion protège bien les données	Accès direct aux routes sans session (401) ; retour arrière après déconnexion
Les opérations de base de données dans tous les cas	Fichier vide, fichier sans attaque, fichier de 5 Mo, même fichier envoyé deux fois, suppression (cascade), base vide
Les entrées dangereuses sont neutralisées	Nom d'utilisateur contenant du code dans un journal ; fichier .exe ; fichier trop gros
La version de production fonctionne comme la version de développement	Checklist complète en pré-production (Gunicorn + front construit)

7.2 Plan de test final

Type	Outil	Critère de réussite
Tests unitaires et d'intégration	pytest + pytest-cov (CI)	100 % des tests verts ; couverture ≥ 80 %
Tests de l'API	Postman	Les 8 routes donnent les codes et le JSON attendus
Tests de bout en bout	Checklist de 13 scénarios	13 / 13 validés en pré-production
Robustesse et performance	cli.py + dataset Loghub	655 146 lignes sans plantage ; 5 Mo en moins de 5 s
Qualité du code	pycodestyle, ESLint	0 erreur

Les résultats sont rassemblés dans `docs/testing/test-report.md` : liens vers les exécutions de la CI, rapport de couverture, export de la collection Postman, checklist datée et mesures de performance. Aucun bug bloquant ou majeur ne doit rester ouvert au tag `v1.0.0`.

8. Livrables à fournir (liens)

Livrable demandé	Lien fourni
Sprint planning	Ce document (Google Doc) · `docs/sprints/sprint-planning.md` · milestones GitHub
Sprint reviews	`docs/sprints/sprint-1-review.md` à `sprint-4-review.md`
Retrospectives	`docs/sprints/sprint-1-retro.md` à `sprint-4-retro.md`
Source repository	`github.com/anil-akr/sentinelssh`
Bug tracking	Issues GitHub filtrées par le label `bug` ; kanban GitHub Projects
Testing evidence and results	Onglet Actions (CI) · `docs/testing/test-report.md` · collection Postman
Production environment	Release GitHub `v1.0.0` · section « Déploiement » du README · vidéo de démonstration en production

> **À confirmer avec ton SWE :** ton environnement de production est volontairement local (l'application écoute sur 127.0.0.1, pour ne pas exposer de données). La preuve proposée est donc la release `v1.0.0`, la procédure de déploiement et une vidéo de l'application en production. Si une adresse accessible en ligne est exigée, il faudra un serveur avec HTTPS : demande-le tôt, car cela ajoute du travail au sprint 4.

9. Préparer la revue technique (MR)

9.1 Checklist « prêt pour la MR »

- L'application fonctionne en production, sans bug bloquant ou majeur.
- Le dépôt GitHub contient tout le code, bien rangé, avec docstrings et commentaires utiles.
- Le README décrit le projet et contient le **schéma d'architecture** et le **diagramme de la base de données** (images dans `docs/images`).
- L'historique Git est propre : branches, PR relues, commits clairs, tags de fin de sprint.
- Les tests passent en direct (`pytest --cov`) et la CI est verte.
- Les documents des Stages 1 à 3, les revues et les rétrospectives sont accessibles par lien.

9.2 Ce que le SWE va évaluer, et où trouver tes réponses

Point évalué	Ce que tu dois savoir expliquer	Référence
Fonctionnement de l'application	Le trajet d'une analyse, de l'envoi du fichier à l'affichage	Stage 3 §2.2 et §4
Ton code et tes fonctionnalités	Chaque classe, chaque route, chaque composant React	Stage 3 §3
Choix techniques	Pourquoi Flask, React, SQLite, sessions, Gunicorn ; pourquoi 3 niveaux de gravité	Stage 3 §7
Base de données	Les 3 tables, la relation 1 → N, la clé étrangère, la cascade, la requête des récidivistes	Stage 3 §3.4
Authentification et sécurité	Hachage vs chiffrement, cookie de session HttpOnly/SameSite, 401, injection SQL, XSS	Conception §13
RBAC	Pourquoi il n'y a qu'un rôle (administrateur) et comment on ajouterait des rôles en V2	Conception §2.4
Tests	Tests unitaires, d'intégration, Postman, checklist ; montrer la couverture	Stage 3 §6.2
Git et GitHub	main / develop / feature, PR, revue, CI, tags	Stage 3 §6.1
Front-end	Composants, props, state, useEffect, routes protégées, responsive	Stage 3 §3.3

> **Méthode de préparation :** pour chaque fichier du projet, entraîne-toi à expliquer en 2 minutes ce qu'il fait et pourquoi il est écrit ainsi. Si une partie a été écrite avec l'aide d'une IA et que tu ne sais pas l'expliquer, reprends-la avant la MR : c'est exactement ce que le SWE vérifiera.

Annexe — Journal des décisions

ID	Date	Décision	Options envisagées	Raison
D1	29/09/2026	Idée retenue : SentinelSSH	Les 6 idées du Stage 1	Meilleur score (26/30) ; défensif, complet, différent de fail2ban
D2	29/09/2026	Analyse à la demande	Temps réel	Faisable en 4 semaines ; le temps réel passe en version future
D3	29/09/2026	Front-end en React (Vite)	HTML/CSS simple, Vue	Technologie JavaScript exigée ; standard le plus répandu
D4	29/09/2026	Back-end en Python/Flask	Node.js/Express, FastAPI, Django	Langage le mieux maîtrisé pour le cœur ; Flask est le plus léger
D5	29/09/2026	Base de données SQLite	PostgreSQL, MongoDB	Usage local, un seul utilisateur, données reliées
D6	29/09/2026	Blocage suggéré, pas automatique	Blocage automatique par iptables	Pas de droits root ; décision humaine ; différence avec fail2ban
D7	30/09/2026	Connexion par un compte administrateur unique	Aucune connexion, plusieurs comptes	Application sécurisée attendue par le titre RNCP ; reste simple
D8	30/09/2026	Mise en production locale avec Gunicorn	Serveurs de développement, Docker	Compétence « déploiement » ; une seule commande pour la démo
D9	01/10/2026	Back-end organisé en classes	Fonctions dans des modules	Consigne du Stage 3 ; une responsabilité par classe ; tests plus simples
D10	01/10/2026	Branches main / develop / feature	Tout sur main	main reste présentable ; chaque fusion passe par une PR et la CI